

Documentatie Proiect Proiectare Grafica

Student:Curea Andrei

January 19, 2026

Contents

1	Prezentarea temei	3
2	Scena	3
2.1	Descrierea scenei si a obiectelor	3
2.2	Functionalitati	3
3	Detalii de implementare	4
3.1	Functii si algoritmi	4
3.1.1	Solutii posibile	4
3.1.2	Motivarea abordarii alese	5
3.2	Modelul grafic (pipeline) si fluxul de randare	5
3.2.1	Pipeline-ul folosit	5
3.2.2	De ce calculez normalMatrix	5
3.2.3	Fluxul complet de randare (frame loop)	6
3.3	Structuri de date	6
3.3.1	La nivel de aplicatie (main.cpp)	6
3.3.2	La nivel de model/mesh	6
3.4	Ierarhia de clase (si cum comunica intre ele)	7
3.4.1	Window	7
3.4.2	Shader	7
3.4.3	Camera	7
3.4.4	Model3D + Mesh	8
4	Detalii concrete pentru cerintele din proiect (legat direct de cod)	8
4.1	Vizualizarea scenei: scalare, translatie, rotatie	8
4.1.1	Transformari in initModels()	8
4.1.2	Aplicarea transformarilor la desenare	8
4.2	Miscarea camerei (tastatura + mouse) + animatii de prezentare	9
4.2.1	Tastatura: processMovement()	9
4.2.2	Up/Down (space si left ctrl)	9
4.2.3	Mouse: mouseCallback()	9
4.2.4	Animatii de prezentare	10
4.3	Surse de lumina (minim doua, in proiect sunt trei)	10
4.3.1	Cum selectez lumina	10
4.3.2	Directional light + umbre	10
4.3.3	Point light (local) cu atenuare	11

4.3.4	Spot light (flashlight) cu cutoff	11
4.4	Moduri de randare: solid, wireframe, poligonal, smooth	11
4.4.1	Poligonal overlay (implementare concreta)	11
4.4.2	Diferenta SOLID vs SMOOTH (implementare in shader)	12
4.5	Texturi si materiale	12
4.5.1	Incarcare materiale din .mtl	12
4.5.2	Incarcare textura in GPU (stb_image)	12
4.5.3	Aplicare textura la desenare	12
4.6	Umbre (Shadow Mapping) - explicatie completa pe implementarea mea . .	13
4.6.1	Ideea (2 pase) [R1]	13
4.6.2	1) Crearea FBO + textura de adancime: initShadowMap()	13
4.6.3	2) Matricea luminii: updateLightSpaceMatrix()	13
4.6.4	3) Depth pass efectiv: renderShadowPass()	14
4.6.5	4) Shading pass + sampling umbre in fragment shader	14
4.6.6	De ce shadow_depth.frag e gol?	14
4.7	Fog (element de fotorealism) + toggle in runtime	15
4.8	Skybox (cubemap) ca fundal realist	15
5	Prezentarea interfetei utilizator / manual de utilizare	15
5.1	Camera	15
5.2	Lumina (moduri)	16
5.3	Umbre	16
5.4	Fog	16
5.5	Vizualizare (moduri randare)	16
5.6	Iesire	16
6	Concluzii si dezvoltari ulterioare	16
6.1	Concluzii	16
6.2	Dezvoltari ulterioare (ce as adauga daca as mai avea timp)	17

1 Prezentarea temei

Tema proiectului a fost realizarea unei scene 3D interactive in OpenGL (pipeline programabil), care sa includa: transformari geometrice (scalare/translatie/rotatie), control camera (tastatura + mouse), animatii, cel putin doua surse de lumina diferite, texturare/materiale, moduri de afisare (solid/wireframe/poligonal/smooth), generarea umbrelor, plus elemente de fotorealism (skybox si ceata).

In implementarea mea, toate aceste cerinte sunt integrate intr-un singur program (main.cpp) care foloseste clase separate pentru fereastra (Window), camera (Camera), shadere (Shader), incarcare si desenare modele (Model3D/Mesh), plus shadere GLSL pentru randare standard, skybox si depth-pass pentru umbre.

Tehnicile utilizate sunt inspirate din laboratoare: shadow mapping [R1], cubemap/skybox [R2], fog [R3], plus documentatia OpenGL si tutoriale tip LearnOpenGL [R8].

2 Scena

2.1 Descrierea scenei si a obiectelor

Scena este inspirata din diferite jocuri video cu tematica dark fantasy, cu obiecte multiple incarcate din fisiere .obj/.mtl. In cod (main.cpp) am definit cate un obiect `gps::Model3D` pentru fiecare element mare din scena: `ground`, `solaire`, `forest`, `bonfire`, `artorias`, `gwyn`, `town`, `castle`, `dragon`, `windmill`. Fiecare are o matrice de model dedicata (`groundModel`, `solaireModel`, etc.), ceea ce imi permite sa controlez pozitia/rotatia/scalarea separat pentru fiecare obiect.

Am ales aceasta structura (model + matrice model) pentru ca:

- e clar ce transformari aplic pe fiecare obiect (usor de prezentat la documentatie)
- pot anima selectiv (ex: bonfire pulse, windmill rotate) fara sa afectez restul scenei
- pot desena aceleasi obiecte in doua pase diferite (normal + shadow depth) cu aceleasi transformari, ceea ce e esential la shadow mapping [R1].

2.2 Functionalitati

Functionalitati implementate (si unde apar in cod):

- Transformari per-obiect (scalare/translatie/rotatie): `initModels()` + folosirea `drawModelWithTra`
- Miscare camera (WASD + up/down) + rotatie cu mouse: `processMovement()`, `mouseCallback()`
- Animatii: `updateBonfireLight(time)` (flicker/jitter + pulsare)
- Surse de lumina: directional + point + in plus spot (flashlight) selectabile din taste: `keyboardCallback()` + shader `basic.frag`
- Moduri de randare: solid, wireframe, poligonal overlay, smooth shading: `renderScene()` + `viewMode` + `glPolygonMode`
- Texturi/materiale: incarcate din .mtl in `Model3D::ReadOBJ()` si aplicate in `Mesh::Draw()`

- Umbre (shadow mapping): `initShadowMap()`, `renderShadowPass()`, `shader shadow_depth.*` + `ShadowFactor()` in `basic.frag`
- Fotorealism: skybox cubemap + sRGB framebuffer + fog (3 moduri) + lumini multiple + umbre [R1][R2][R3]

3 Detalii de implementare

3.1 Functii si algoritmi

3.1.1 Solutii posibile

Pentru fiecare cerinta, au existat mai multe optiuni:

1) Camera si miscarea in scena

- camera orbitala (in jurul centrului) – simplu, dar limitat
- camera FPS/free-fly (pozitie + directie + yaw/pitch) – mai naturala pentru explorare

2) Iluminare

- Phong (specular in fragment) vs Blinn-Phong (half-vector)
- lumina directionala vs punctiforma vs spot (flashlight)

3) Umbre

- shadow mapping [R1] (2 pase: depth + shading)
- shadow volumes (mai complex in timp real)

4) Skybox

- fundal 2D simplu (nu realist)
- cubemap/skybox (6 fete), corect pentru scene 3D [R2]

5) Fog

- linear
- exponential
- exponential squared (exp2) – cel mai “natural” vizual [R3]

3.1.2 Motivarea abordarii alese

Am ales:

- camera free-fly pentru ca permite explorare naturala cu WASD + mouse
- 3 tipuri de lumina (directional + point + spot) pentru a bifa cerinta de minim 2 si pentru a demonstra comparativ moduri diferite de iluminare
- shadow mapping pentru ca se integreaza bine cu OpenGL modern si este cerut explicit in laboratorul de umbre [R1]
- skybox cubemap pentru realism si pentru ca se potriveste cu scena mare (evita “marginii” vizuale) [R2]
- fog (3 moduri) pentru efect de atmosfera + adancime, si ca element de “fotorealism” din cerinta [R3]

3.2 Modelul grafic (pipeline) si fluxul de randare

3.2.1 Pipeline-ul folosit

Aplicatia foloseste pipeline programabil:

- Vertex Shader: calculeaza pozitia finala (MVP), normale, coordonate UV, plus coordonate in spatiul luminii (pentru umbre)
- Fragment Shader: aplica texturare, iluminare (in functie de modul ales), shadow factor, fog, apoi scrie culoarea finala

In `basic.vert` se vede clar transformarea:

$$worldPos = M \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}, \quad gl_Position = P \cdot V \cdot worldPos$$

si se calculeaza si:

$$FragPosLightSpace = LightSpace \cdot worldPos$$

pentru shadow mapping [R1].

3.2.2 De ce calculez normalMatrix

In `drawModelWithTransform()` calculez:

$$normalMatrix = (V \cdot M)^{-T}$$

Implementare: `glm::mat3 nm = mat3(transpose(inverse(view * modelMat)));`

Motiv: daca aplic scalare neuniforma sau transformari complexe, normalele nu mai sunt corecte doar cu model matrix. Folosind inversa transpusa, obtin normale corecte pentru iluminare.

3.2.3 Fluxul complet de randare (frame loop)

In `main()` se vede loop-ul principal:

- calculez `deltaTime` din `glfwGetTime()` pentru miscari consistente
- apelez `updateBonfireLight(t)` (animatie + lumina locala)
- apelez `processMovement()` (camera)
- apelez `renderScene()` (randare finala)

Aceasta separare (update input/animatie/randare) este clasica pentru jocuri si proiecte 3D.

3.3 Structuri de date

3.3.1 La nivel de aplicatie (main.cpp)

- `pressedKeys[1024]`: buffer de stari pentru taste; eu nu misc camera direct in callback, ci tin state si aplic in `processMovement()`. Avantaj: miscarea devine continua, nu doar la eveniment.
- `glm::mat4` pentru view, projection si cate un model per obiect.
- enum `LightMode` si enum `ViewMode`: imi organizeaza modurile de lucru clar (lumina si randare).
- variabile de timp: `lastFrame`, `deltaTime` pentru consistenta la input si animatii.
- uniform locations (`GLint ...Loc`): le citesc o singura data in `initUniforms()`, ca sa nu fac `glGetUniformLocation` la fiecare draw (eficient).

3.3.2 La nivel de model/mesh

Structura Vertex In `Mesh.hpp` am: `Position`, `Normal`, `TexCoords`. Acestea core-spond exact layout-ului din `basic.vert`:

- location 0: position
- location 1: normal
- location 2: texcoords

Buffers (VAO/VBO/EBO) In `Mesh::setupMesh()` creez:

- VAO: retine configuratia atributelor
- VBO: contine vertecsii (struct `Vertex`)
- EBO: contine indicii (triangulare)

De aceea in `Draw()` este suficient: `glBindVertexArray(VAO); + glDrawElements(...)`.

Cache de texturi In `Model3D` tin `loadedTextures`. In `LoadTexture()` verific daca o textura a fost deja incarcata: daca da, refolosesc ID-ul OpenGL. Motiv: multe mesh-uri pot folosi aceeasi textura si nu vreau duplicare de memorie.

Grupare pe material In `Model3D::ReadOBJ()` folosesc `map<int, vector<Vertex>>` si `map<int, vector<GLuint>>` pentru a grupa geometria pe material id. Motiv: un fisier `.obj` poate avea materiale diferite pe fete diferite. Eu creez cate un `Mesh` pentru fiecare material, cu setul lui de texturi.

3.4 Ierarhia de clase (si cum comunica intre ele)

3.4.1 Window

`gps::Window` se ocupa de:

- init GLFW + context OpenGL 4.1 core
- init GLEW (non-Apple)
- setari: sRGB framebuffer, MSAA, HiDPI

Eu am activat sRGB (`GLFW_SRGB_CAPABLE + glEnable(GL_FRAMEBUFFER_SRGB)`). Asta imbunatateste realismul culorilor atunci cand folosesc texturi in spatiul sRGB.

3.4.2 Shader

`gps::Shader`:

- citeste fisierele `.vert/.frag`
- compileaza shaderele
- link-uieste programul
- afiseaza log la erori (compile/link)

Este folosit de `Model3D/Mesh` ca sa deseneze cu programul activ.

3.4.3 Camera

`gps::Camera` tine: `Position`, `Front`, `Right`, `Up`. Functii cheie:

- `getViewMatrix()` -> `glm::lookAt(pos, pos+front, up)`
- `move(dir, speed)` -> deplasare in spatiul camerei (pe front/right)
- `rotate(pitch,yaw)` -> actualizeaza front pe baza unghiurilor (limit pitch la +/-89)

In main, camera este controlata astfel:

- WASD: `myCamera.move(...)`
- mouse: `myCamera.rotate(...)` (sensibilitate scalata)
- up/down: in `processMovement()` extrag pozitia din view si reconstruiesc camera ca sa urce/coboare

3.4.4 Model3D + Mesh

Model3D incarca .obj prin tinyobjloader [R4]. Mesh este unitatea desenabila, cu buffer-ele si texturile. Flux:

Model3D :: LoadModel → ReadOBJ → creezMesh(vertices, indices, textures) → Mesh :: Draw

In `Mesh::Draw()`:

- activez fiecare textura (`glActiveTexture(GL_TEXTURE0+i)`)
- setez uniform-ul sampler cu numele `type` (ex: `diffuseTexture`)
- desenez elementele
- dezactivez texturile

Asta explica de ce in `initShaders()` eu setez explicit: `diffuseTexture` pe unitatea 0 si `specularTexture` pe 1 (chiar daca in fragment folosesc doar diffuse in varianta curenta).

4 Detalii concrete pentru cerintele din proiect (legat direct de cod)

4.1 Vizualizarea scenei: scalare, translatie, rotatie

4.1.1 Transformari in `initModels()`

In `initModels()` pentru fiecare obiect:

- incarc modelul: `LoadModel("models/.../x.obj")`
- construiesc matricea model: `translate, rotate, scale`

Exemple:

- Ground: doar `scale(10)` ca sa devina “teren” mare.
- Bonfire: `translate(0.8, 0.0, -0.8) + scale(10)`.
- Windmill: `translate(10, 0, 60) + scale(10) + rotate(-30 deg)` ca sa “orientez” obiectul spre scena (asta corespunde cerintei de rotatie corecta).

Motivare: am preferat sa fac transformari in cod, nu sa modific permanent mesh-ul, ca sa pot ajusta rapid scena si sa pot explica clar in documentatie.

4.1.2 Aplicarea transformarilor la desenare

In `drawModelWithTransform mdl, modelMat`:

- trimit uniform-ul `model` cu `modelMat`
- trimit `view` si `projection`
- calculez normal matrix si o trimit

Asta inseamna ca fiecare obiect este desenat cu transformarea lui, dar foloseste aceeasi camera si aceeasi proiectie.

4.2 Miscarea camerei (tastatura + mouse) + animatii de prezentare

4.2.1 Tastatura: processMovement()

In `processMovement()`:

- calculez `speed = cameraMoveSpeed * deltaTime`
- daca apas W/S/A/D: miscare in planul camerei (front/right)

De ce `deltaTime`? Ca miscarea sa fie egala indiferent de FPS (daca am 30 FPS sau 144 FPS).

4.2.2 Up/Down (space si left ctrl)

Eu am implementat urcare/coborare (free-fly) astfel:

- extrag `pos` si `front` din view matrix (inversa view-ului)
- modific `pos.y`
- reconstruiesc camera: `myCamera = Camera(pos, pos + front, up)`

Am ales varianta asta pentru ca in implementarea mea Camera nu are `setPosition` direct, dar pot recrea obiectul fara bug-uri, pastrand directia.

4.2.3 Mouse: mouseCallback()

In `mouseCallback()`:

- calculez offset x/y fata de pozitia anterioara
- scalez cu `mouseSensitivity`
- apelez `myCamera.rotate(pitch,yaw)`

In `Camera::rotate()` eu:

- calculez yaw/pitch curent din vectorul front
- adaug offset
- limitez pitch la +/- 89 grade ca sa evit "gimbal flip"
- reconstruiesc frontul cu trigonometrie (cos/sin)
- refac right/up cu produse vectoriale

4.2.4 Animatii de prezentare

In proiectul meu, animatia “de prezentare” este implementata prin dependenta de timp (`glfwGetTime()`): exemplul cel mai clar este bonfire (lumina si “pulsul” obiectului).

In `updateBonfireLight(t)`:

- extrag pozitia de baza a bonfire-ului din matricea `bonfireBaseModel`
- adaug jitter pe x/z folosind `sin/cos` (simuleaza flacara)
- calculez flicker in intensitate tot cu `sin` pe frecvente diferite
- actualizez `pointLightPos` si `pointLightColor` in timp real
- scalez usor modelul (pulse) ca sa se vada animatia pe obiect (nu doar in lumina)

De ce am ales `sin/cos`? Pentru ca e usor, stabil, periodic si imi da o variatie “naturala” fara sa folosesc noise complex.

4.3 Surse de lumina (minim doua, in proiect sunt trei)

In proiect am 3 moduri selectabile:

- **Directional** (tasta 1): `lightDir`, `lightColor`
- **Point light** (tasta 2): pozitionata la bonfire, cu flicker
- **Spot light / flashlight** (tasta 3): atasata camerei (pozitie = `cameraPos`, directie = `cameraFront`)

4.3.1 Cum selectez lumina

In `keyboardCallback()` schimb `lightMode`. In `drawModelWithTransform()` trimit uniform-ul `lightMode`. In `basic.frag` aleg: `ComputeDirectional` / `ComputePoint` / `ComputeSpot`.

4.3.2 Directional light + umbre

In `ComputeDirectional()`:

- $L = \text{normalize}(-\text{lightDir})$
- calculez $\text{diffuse} = \max(\text{dot}(N, L), 0)$
- calculez specular cu `reflect`
- daca `useShadows==1`: calculez shadow factor (0/1) si multiplic doar `diffuse+specular`

Asta inseamna ca in umbra ramane doar ambient (realist).

4.3.3 Point light (local) cu atenuare

In `ComputePoint()`:

- vector spre lumina: `pointLightPos - FragPosWorld`
- `distanta = length()`
- atenuare:

$$att = \frac{1}{c + l \cdot d + q \cdot d^2}$$

- lumina finala = (ambient + diffuse + specular) * att

Eu am setat valori tipice pentru atenuare (0.09, 0.032) ca sa se simta local.

4.3.4 Spot light (flashlight) cu cutoff

In `ComputeSpot()`:

- calculez `theta = dot(L, -spotDir)`
- folosesc inner/outer cutoff si fac interpolare lineara (smooth edge):

$$intensity = clamp\left(\frac{\theta - outer}{inner - outer}, 0, 1\right)$$

- aplic intensitatea doar pe diffuse+specular (ambient ramane constant)

Flashlight-ul este “legat” de camera in `drawModelWithTransform()`: `spotLightPos = cameraPos`, `spotLightDir = cameraFront`.

4.4 Moduri de randare: solid, wireframe, poligonal, smooth

In proiect am `ViewMode`:

- **Wireframe**: `glPolygonMode(GL_LINE)`
- **Poligonal overlay**: desenez intai fill, apoi desenez wire peste (cu polygon offset)
- **Solid** (faceted / flat look): nu schimb polygon mode, dar schimb modul de normal in shader
- **Smooth**: shading normal (normalele din model)

4.4.1 Poligonal overlay (implementare concreta)

In `renderScene()` pentru `VIEW_POLYGONAL`:

- desenez fill normal
- activez `GL_POLYGON_OFFSET_LINE` cu offset negativ ca sa nu am z-fighting
- trec in `GL_LINE` si desenez inca o data scena, dar cu `uOverrideColorEnabled=1` si `uOverrideColor = (0,0,0)`
- dezactivez culling temporar ca sa se vada muchiiile si pe spate (mai clar ca “poligonal”)

Asta e un mod standard de a obtine efectul “wire overlay” fara artefacte.

4.4.2 Diferenta SOLID vs SMOOTH (implementare in shader)

Eu nu fac SOLID doar prin polygon mode, ci schimb shading-ul:

- in VIEW_SMOOTH: folosesc `NormalWorld` din vertex shader (interpolata).
- in VIEW_SOLID (si POLYGONAL): setez `uShadingMode=1` si in fragment calculez o normala per-fragment faceted:

$$N = \text{normalize}(\text{cross}(dFdx(\text{FragPosWorld}), dFdy(\text{FragPosWorld})))$$

Practic, asta forteaza o normala “de fata”, ca sa se vada muchiile mai tare si diferenta fata de smooth.

4.5 Texturi si materiale

4.5.1 Incarcare materiale din .mtl

In `Model3D::ReadOBJ()` folosesc `tinyobjloader` [R4] care imi da: `materials[matId].diffuse_texname`, `specular_texname`, `ambient_texname`.

Pentru fiecare material:

- daca exista `diffuse_texname`: incarc textura si o adaug cu type `"diffuseTexture"`
- daca exista `specular_texname`: incarc cu type `"specularTexture"`

Apoi creez un `Mesh(vertices, indices, textures)`.

4.5.2 Incarcare textura in GPU (stb_image)

In `ReadTextureFromFile()`:

- citesc imaginea cu `stb_image` [R5] fortand 4 canale (RGBA)
- fac flip vertical manual (pentru ca coordonatele UV din obj sunt invers pe Y fata de OpenGL)
- generez textura OpenGL, incarc cu `glTexImage2D`
- folosesc `GL_SRGB` ca internal format pentru corectie de gamma (in combinatie cu framebuffer sRGB)
- generez mipmap si setez filtrare `GL_LINEAR_MIPMAP_LINEAR`

Motivare: mipmap + filtrare buna = texturi mai calitative la distanta, fara shimmering.

4.5.3 Aplicare textura la desenare

In `Mesh::Draw()`:

- pentru fiecare textura, activez unitatea (0,1,2...)
- setez uniform-ul sampler cu numele tipului (`diffuseTexture` etc.)
- bind textura, apoi draw

In `basic.frag` folosesc: `albedo = texture(diffuseTexture, TexCoord).rgb`; Deci culoarea de baza vine direct din textura.

4.6 Umbre (Shadow Mapping) - explicatie completa pe implementarea mea

4.6.1 Ideea (2 pase) [R1]

Shadow mapping are doua pase:

1. Depth pass: randare din perspectiva luminii intr-o textura de adancime (shadowDepthTex)
2. Shading pass: randare normala din perspectiva camerei; pentru fiecare fragment verific daca este in umbra comparand adancimea lui in spatiul luminii cu valoarea din shadow map

4.6.2 1) Crearea FBO + textura de adancime: initShadowMap()

In `initShadowMap()`:

- creez FBO: `glGenFramebuffers`
- creez textura `shadowDepthTex` cu `GL_DEPTH_COMPONENT`, rezolutie 2048x2048
- setez filtre `NEAREST` (corect pentru depth compare clasic)
- setez wrap la `CLAMP_TO_BORDER` si `borderColor = 1` (in afara frustum-ului sa nu creeze umbre false)
- atasez textura la `GL_DEPTH_ATTACHMENT`
- dezactivez color outputs: `glDrawBuffer(GL_NONE)`

4.6.3 2) Matricea luminii: updateLightSpaceMatrix()

In `updateLightSpaceMatrix()`:

- aleg un `sceneCenter` fix (0,0,0)
- calculez o pozitie a luminii directionale: `lightPos = center - lightDir * 80`
- `lightView = lookAt(lightPos, center, up)`
- `lightProj = ortho(...)` (ortografica, corect pentru directional light)

Rezultat:

$$LightSpace = lightProj \cdot lightView$$

Am folosit ortho size mare (90) si near/far (1..200) ca sa includ scena mare (teren + cladiri).

4.6.4 3) Depth pass efectiv: `renderShadowPass()`

In `renderShadowPass()`:

- setez viewport la rezolutia shadow map (2048)
- bind FBO si clear depth
- schimb culling pe front face (`glCullFace(GL_FRONT)`) ca sa reduc shadow acne (o tehnica practica folosita des)
- folosesc `depthShader` (`shadow_depth.vert/frag`)
- desenez toate obiectele prin `drawAllObjectsDepth()`
- revin culling back si revin pe framebuffer default + viewport normal

4.6.5 4) Shading pass + sampling umbre in fragment shader

In `renderScene()`:

- daca `useShadows` si modul de lumina e `directional`: apelez `renderShadowPass()`
- bind `shadowDepthTex` pe unitatea 2 (conform `initShaders`)
- desenez obiectele normal cu `drawAllObjects()`

In `basic.frag`, functia `ShadowFactor()`:

- proiectez coordonata din light space: `projCoords = fragLS.xyz / fragLS.w`
- o aduc in `[0,1]`: `projCoords = projCoords*0.5+0.5`
- daca `projCoords.z > 1`: nu e umbra (in afara frustum-ului)
- `closest = texture(shadowMap, projCoords.xy).r`
- `current = projCoords.z`
- aplic bias dependent de unghi:

$$bias = \max(0.0015(1 - \langle N, L \rangle), 0.0008)$$

- daca `current - bias > closest` = umbra (return 0), altfel luminat (return 1)

Asta rezolva shadow acne si supra-sampling in mod similar cu laboratorul [R1].

4.6.6 De ce `shadow_depth.frag` e gol?

In pass-ul de umbre ma intereseaza doar buffer-ul de adancime. OpenGL scrie automat adancimea, deci fragment shader poate fi gol (nu am iesire de culoare). E o practica normala la depth-only pass [R1].

4.7 Fog (element de fotorealism) + toggle in runtime

In proiect am:

- `fogEnabled` toggle (tasta F)
- `fogMode` cycling (tasta M): 0=linear, 1=exp, 2=exp2

In shader, `ApplyFog(col)`:

- `distanta = length(cameraPos - FragPosWorld)`
- calculez `fogT` in functie de mod:
 - linear: `clamp((end-dist)/(end-start))`
 - exp: `exp(-density*dist)`
 - exp2: `exp(-(density*dist)2)`
- culoare finala: `mix(fogColor, col, fogT)`

Am ales exp2 ca default (`fogMode=2`) pentru ca vizual arata cel mai natural [R3].

4.8 Skybox (cubemap) ca fundal realist

Skybox-ul este implementat fara clasa separata, direct in main:

- `initSkybox()` creeaza un cub (36 verteci) in VAO/VBO
- incarc 6 imagini TGA intr-un cubemap texture (`loadCubemap`)
- in `drawSkybox()` desenez la final cu depth func LEQUAL

In `drawSkybox()` folosesc `viewNoTrans = mat4(mat3(view))` ca sa scot translatarea camerei. Motiv: skybox-ul trebuie sa para “infinite”, adica sa se roteasca odata cu camera, dar sa nu se deplaseze.

In `skybox.vert` fac `gl_Position = pos.xyww`; ca sa tin skybox-ul in fundal (depth 1.0).

Implementarea urmeaza ideea din laboratorul de cubemaps [R2].

5 Prezentarea interfetei utilizator / manual de utilizare

Aplicatia este controlata din tastatura si mouse:

5.1 Camera

- W/A/S/D: deplasare in fata/stanga/spate/dreapta
- Mouse: rotatie camera (yaw/pitch)
- Space: urcare
- Left Ctrl: coborare
- Scroll: modifica FOV (zoom), intre 20 si 80 grade

5.2 Lumina (moduri)

- 1: Directional light (global)
- 2: Point light (bonfire local + flicker)
- 3: Spot light (flashlight atasat camerei)
- N: enable/disable lighting (daca e OFF, ramane doar textura + fog)

5.3 Umbre

- H: enable/disable shadows (shadow mapping activ doar la directional)

5.4 Fog

- F: enable/disable fog
- M: schimba modul (LINEAR / EXP / EXP2)

5.5 Vizualizare (moduri randare)

- 4: Solid (faceted/flat normal, fara wireframe)
- 5: Wireframe
- 6: Poligonal (fill + wire overlay negru)
- 7: Smooth (normale interpolate)

5.6 Iesire

- ESC: inchide aplicatia

6 Concluzii si dezvoltari ulterioare

6.1 Concluzii

Am implementat o scena 3D complexa in OpenGL, care bifeaza cerintele:

- transformari complete (scalare, translatie, rotatie) pentru fiecare obiect
- camera controlata din tastatura + mouse + up/down
- animatii (bonfire pulse + flicker/jitter; scena poate fi extinsa cu windmill blade separat daca modelul are mesh-uri distincte)
- minim 2 surse de lumina (in proiect sunt 3: directional, point, spot)
- moduri de randare: solid, wireframe, poligonal overlay, smooth
- texturare/materiale: .mtl + stb_image, mipmaps, sRGB
- umbre: shadow mapping complet (FBO + depth texture + compare in shader)
- fotorealism: skybox cubemap + sRGB + fog + umbre + iluminare multipla

6.2 Dezvoltari ulterioare (ce as adauga daca as mai avea timp)

- PCF (Percentage Closer Filtering) pentru umbre mai soft (anti-aliasing in shadow edges) [R1][R8]
- cascaded shadow maps pentru scena foarte mare (calitate mai buna la distanta)
- animatie separata pentru componente (ex: pale windmill pe mesh separat) - daca separ in Blender si reexport [R6]
- efecte meteo: particule pentru ploaie, miscari usoare de “vant” pe vegetatie (vertex shader)
- coliziuni simple camera vs obiecte (AABB/OBB) ca sa nu trec prin cladiri

Referinte

- R1 Laborator Shadow Mapping (OpenGL 4) - tehnica depth map + bias + clamp to border
- R2 Laborator Cubemaps / Skybox
- R3 Laborator Fog + blending/discard
- R4 tinyobjloader (incarcare .obj/.mtl) - folosit in Model3D
- R5 stb_image (incarcare texturi) - folosit in Model3D
- R6 Blender (procesare/export modele, orientare, separare mesh-uri)
- R7 Documentatia OpenGL/GLFW/GLEW (context, callbacks, input)
- R8 LearnOpenGL (shadow mapping, skybox, lighting) <https://learnopengl.com>